VS Code Shortcut Cheatsheet — Python, Django, HTML, CSS
Windows / Linux | Practical Developer Reference

Purpose: Faster coding, debugging, formatting, navigation, and Django development.

> Note: Shortcuts can vary by operating system, keyboard layout, extensions, and custom VS Code keybindings. Use `Ctrl + K`, then `Ctrl + S` to open Keyboard Shortcuts and verify any binding.

PART 1: Python / Django Shortcuts

No.	Action	Shortcut	Purpose
1	Run Without Debugging	`Ctrl + F5`	Run the current launch configuration without debugging
2	Start Debugging	`F5`	Start or continue a debugging session
3	Stop Debugging	`Shift + F5`	Stop the current debugging session
4	Quick Fix / Code Action	`Ctrl + .`	Show available fixes and code actions
5	Rename Symbol	`F2`	Rename a variable, function, class, or supported symbol
6	Format Document	`Shift + Alt + F`	Format the current file using the configured formatter
7	Trigger Suggestions	`Ctrl + Space`	Show code completion suggestions
8	Select Next Matching Occurrence	`Ctrl + D`	Add the next matching selection
9	Go to Definition	`F12`	Open the definition of a symbol
10	Peek Definition	`Alt + F12`	Preview a symbol's definition
11	Find in File	`Ctrl + F`	Search within the current file
12	Replace in File	`Ctrl + H`	Find and replace text
13	Go to Line	`Ctrl + G`	Jump to a specific line
14	Command Palette	`Ctrl + Shift + P`	Search for VS Code commands
15	Save File	`Ctrl + S`	Save the current file

16	Save All	`Ctrl + K`, then `S`	Save all files
17	Toggle Line Comment	`Ctrl + /`	Comment or uncomment selected lines
18	Move Line Up/Down	`Alt + ↑` / `Alt + ↓`	Move the current line
19	Copy Line Up/Down	`Shift + Alt + ↑` / `Shift + Alt + ↓`	Duplicate the current line
20	Multi-Cursor	`Alt + Click`	Place multiple cursors

Python Formatting Setup

Install the Python extension and configure a formatter such as Black.

Install Black:

```bash
python -m pip install black
```

Format a Python file:

```bash
python -m black students/views.py
```

You can also configure a formatter in VS Code settings and use `Shift + Alt + F`.

Django Development Tips

- Use `F12` to inspect model, function, or class definitions.
- Use `Ctrl + .` to see available quick fixes.
- Use `Ctrl + Space` to trigger completion suggestions.
- Use the integrated terminal for Django management commands.
- Run tests frequently instead of relying only on manual browser testing.

Example commands:

```bash
python manage.py runserver
python manage.py check
python manage.py test
python manage.py makemigrations
python manage.py migrate
```

Important: `Ctrl + F5` runs the current launch configuration. It does not automatically mean "run the current Django project" unless the appropriate launch configuration is set.

PART 2: HTML / CSS Shortcuts

No.	Action	Shortcut	Purpose
1	Emmet Expansion	Type abbreviation, then `Tab`	Expand supported HTML/CSS abbreviations
2	Duplicate Line	`Shift + Alt + ↓`	Duplicate the current line
3	Move Line	`Alt + ↑` / `Alt + ↓`	Move a line up or down
4	Format Document	`Shift + Alt + F`	Format HTML/CSS using the configured formatter
5	Toggle Comment	`Ctrl + /`	Comment or uncomment code
6	Select All Occurrences	`Ctrl + Shift + L`	Select every matching occurrence
7	Find	`Ctrl + F`	Search the current file
8	Replace	`Ctrl + H`	Replace text in the current file
9	Toggle Word Wrap	`Alt + Z`	Wrap long lines visually
10	Open File	`Ctrl + P`	Quickly open a file by name

Emmet Examples

Type:

```text
!
```

Then press `Tab` in an HTML file to expand the standard HTML boilerplate when Emmet is available.

Type:

```text
ul>li*3
```

Press `Tab` to expand:

```html
<ul>
    <li></li>
    <li></li>
    <li></li>
</ul>
```

Type:

```text
div.container>h1+p
```

Expand:

```html
<div class="container">
    <h1></h1>
    <p></p>
</div>
```

Important Correction: Wrap With Abbreviation

The standard VS Code Emmet action is **Emmet: Wrap with Abbreviation**. Its shortcut is not universally `Ctrl + Shift + A`.

To use it:

1. Select the HTML you want to wrap.
2. Open the Command Palette using `Ctrl + Shift + P`.
3. Search for `Emmet: Wrap with Abbreviation`.
4. Enter an abbreviation such as `div.container`.
5. Press Enter.

You can assign your own keyboard shortcut in VS Code Keyboard Shortcuts.

PART 3: Django Template Shortcuts and Snippets

Django templates use special syntax inside HTML.

Common Template Syntax

Variables

```django
{{ student.name }}
{{ student.email }}
```

Conditions

```django
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}</p>
{% else %}
    <p>Please log in.</p>
{% endif %}
```

Loops

```django
{% for student in students %}
```

```
    <p>{{ student.name }}</p>
{% empty %}
    <p>No students found.</p>
{% endfor %}
```

Template Inheritance

```django
{% extends "base.html" %}

{% block content %}
    <h1>Student Page</h1>
{% endblock %}
```

CSRF Token

```django
<form method="post">
    {% csrf_token %}
    <button type="submit">Submit</button>
</form>
```

Include a Reusable Template

```django
{% include "includes/navbar.html" %}
```

Load Static Files

```django
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Template Comment

Shortcut:

```text
Ctrl + /
```

This is VS Code's general line-comment shortcut. The exact comment syntax generated depends on the language mode and available extensions.

For Django template comments, the correct syntax is:

```django
{# This is a single-line template comment. #}
```

For a multi-line comment:

```django
{% comment %}
This is a multi-line template comment.
{% endcomment %}
```

Do not assume that `Ctrl + /` always generates Django template comment syntax. Check the resulting code.

Create Custom Django Snippets

VS Code supports user-defined snippets.

1. Open the Command Palette.
2. Search for `Snippets: Configure User Snippets`.
3. Choose HTML or create a suitable custom snippet file.
4. Add snippets for commonly used Django template structures.

Example snippet definitions for an HTML snippet file:

```json
{
    "Django CSRF Form Token": {
        "prefix": "dcsrf",
        "body": [
            "{% csrf_token %}"
        ],
        "description": "Insert Django CSRF token"
    },
    "Django For Loop": {
        "prefix": "dfor",
        "body": [
            "{% for ${1:item} in ${2:items} %}",
            "    $0",
            "{% empty %}",
            "    <p>No items found.</p>",
            "{% endfor %}"
        ],
        "description": "Django template for loop"
    },
    "Django If Block": {
        "prefix": "dif",
        "body": [
            "{% if ${1:condition} %}",
            "    $0",
            "{% else %}",
            "    ",
            "{% endif %}"
        ],
        "description": "Django template if block"
    },
    "Django Content Block": {
        "prefix": "dblock",
        "body": [
            "{% block content %}",
            "    $0",
            "{% endblock %}"
        ],
        "description": "Django template content block"
    }
}
```

Note: Template completion and syntax highlighting may require a Django template extension. Choose an extension compatible with your project's language mode.

PART 4: VS Code Terminal Shortcuts

No.	Action	Windows / Linux	macOS
1	Toggle Integrated Terminal	`Ctrl + ``	`Control + ``
2	Create New Terminal	`Ctrl + Shift + ``	`Control + Shift + ``
3	Clear Terminal	`Ctrl + L` in many shells	`Ctrl + L` in many shells
4	Open Command Palette	`Ctrl + Shift + P`	`Cmd + Shift + P`
5	Open Settings	`Ctrl + ,`	`Cmd + ,`
6	Open Keyboard Shortcuts	`Ctrl + K`, then `Ctrl + S`	`Cmd + K`, then `Cmd + S`

The backtick key is usually located below Esc, but keyboard layouts vary. Use the Command Palette if the shortcut differs on your keyboard.

Common Django Terminal Commands

Activate a virtual environment first.

Windows PowerShell:

```powershell
.\.venv\Scripts\Activate.ps1
```

Windows Command Prompt:

```bat
.venv\Scripts\activate.bat
```

```

Linux / macOS:

```bash
source .venv/bin/activate
```

Start the server:

```bash
python manage.py runserver
```

Create an application:

```bash
python manage.py startapp students
```

Generate migrations:

```bash
python manage.py makemigrations
```

Apply migrations:

```bash
python manage.py migrate
```

Run tests:

```bash
python manage.py test
```

Create a superuser:
```

```bash
python manage.py createsuperuser
```

PART 5: Django Best Practices – Professional Guide

5.1 Scalable Project Structure

Use separate apps for logically distinct features.

Example:

```text
config/
├── settings/
│   ├── base.py
│   ├── development.py
│   └── production.py
├── urls.py
├── wsgi.py
└── asgi.py

accounts/
├── models.py
├── views.py
├── forms.py
├── urls.py
└── tests.py

products/
├── models.py
├── views.py
├── services.py
├── urls.py
└── tests.py

orders/
```

```
├── models.py
├── views.py
├── services.py
├── urls.py
└── tests.py

templates/
static/
manage.py
```

Best practices:

- Separate features by responsibility.
- Keep URL configurations organized.
- Use clear names.
- Add `services.py` when it makes business logic easier to reuse or test.
- Avoid creating unnecessary abstractions in small applications.
- Keep application boundaries understandable.

বাংলা: প্রতিটি app-কে নির্দিষ্ট দায়িত্ব দেওয়া ভালো। তবে ছোট project-এ অতিরিক্ত folder বা layer তৈরি করলে complexity বাড়তে পারে।

5.2 Settings Management

For larger projects, separate shared and environment-specific configuration.

- `base.py`: Shared settings.
- `development.py`: Development configuration.
- `production.py`: Production configuration.

Never commit production secrets.

Use environment variables or a secrets manager for:

- `SECRET_KEY`
- Database credentials
- Email credentials

- API tokens
- Payment gateway credentials

Example:

```python
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
```

Do not commit real credentials to Git.

5.3 Custom User Model

For a new project, consider defining a custom user model before the first migration.

Example:

```python
# accounts/models.py

from django.contrib.auth.models import AbstractUser



class User(AbstractUser):
    pass
```

Register it in settings before running initial migrations:

```python
AUTH_USER_MODEL = "accounts.User"
```

Then create and apply migrations:

```bash
```

```
python manage.py makemigrations
python manage.py migrate
```

Even if the model initially adds no fields, a custom user model gives the project more flexibility later.

Important:

- Decide on the user model at the beginning of a new project.
- Changing `AUTH_USER_MODEL` after tables and relationships have been created can require complex data and schema migration work.
- Reference the user model with `settings.AUTH_USER_MODEL` in model relationships.

Example:

```python
from django.conf import settings
from django.db import models



class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )
```

5.4 Fat Models, Thin Views — With Judgment

A useful principle is to keep views focused on HTTP request and response handling.

However, "put everything in models" is not a universal rule.

Good responsibilities:

- Models: Data structure, constraints, and behavior closely tied to an entity.
- Forms/serializers: Input validation and transformation.
- Services: Multi-step business operations when useful.
- Views: HTTP flow, permissions, and response generation.
- Selectors/query functions: Reusable, complex data retrieval where helpful.

Example:

```python
# products/services.py

from django.db import transaction

from .models import Product



@transaction.atomic
def create_product(*, name, price):
    return Product.objects.create(
        name=name,
        price=price,
    )
```

View:

```python
from django.shortcuts import redirect, render

from .forms import ProductForm
from .services import create_product



def product_create(request):
    if request.method == "POST":
        form = ProductForm(request.POST)
```

```
        if form.is_valid():
            create_product(
                name=form.cleaned_data["name"],
                price=form.cleaned_data["price"],
            )
            return redirect("products:list")
    else:
        form = ProductForm()

    return render(
        request,
        "products/product_form.html",
        {"form": form},
    )
```

This example assumes `ProductForm`, the `Product` model, and the named URL exist. For a `ModelForm`, `form.save()` may be the simplest appropriate solution.

5.5 Database Optimization

Avoid N+1 Queries

Without optimization:

```python
orders = Order.objects.all()

for order in orders:
    print(order.customer.name)
```

If each order accesses its customer separately, the application may execute many extra queries.

For a foreign key or one-to-one relationship:

```python
orders = Order.objects.select_related("customer")
```

```

For reverse foreign key or many-to-many relationships:

```python
orders = Order.objects.prefetch_related("items")
```

Use the method that matches the relationship type.

Add Indexes When Appropriate

```python
from django.db import models



class Product(models.Model):
    name = models.CharField(max_length=200)
    sku = models.CharField(
        max_length=50,
        unique=True,
    )
    created_at = models.DateTimeField(
        auto_now_add=True,
    )

    class Meta:
        indexes = [
            models.Index(fields=["created_at"]),
        ]
```

A `unique=True` constraint generally creates a database-level uniqueness constraint and an associated index on supported databases.

Do not add indexes indiscriminately. Indexes consume storage and can slow writes. Use query plans and real workload measurements to decide.

Inspect Queries
```

Use Django Debug Toolbar during development where appropriate. Use `QuerySet.explain()` to inspect database query plans.

```python
queryset = Product.objects.filter(
    name__icontains="phone"
)

print(queryset.explain())
```

The precise output depends on the database backend.

5.6 Security Best Practices

- Set `DEBUG=False` in production.
- Keep `SECRET_KEY` secret.
- Configure `ALLOWED_HOSTS`.
- Use HTTPS.
- Preserve CSRF protection for browser forms.
- Validate input on the server.
- Enforce authorization in views, services, or other server-side layers.
- Keep dependencies updated.
- Avoid unsafe SQL string construction.
- Do not mark untrusted HTML as safe.
- Protect private uploads.
- Store passwords with Django's password hashing utilities.

5.7 Authentication Workflow

Use Django's built-in authentication framework where possible.

For browser-based applications, session authentication is often appropriate.

For APIs, choose an authentication mechanism that matches the client and threat model.

JWT is not automatically more secure than session authentication. Consider token storage, expiration, revocation, transport security, and CSRF/XSS risks.

For OTP and email verification:

- Generate tokens securely.
- Use expiration times.
- Rate-limit verification attempts.
- Avoid leaking whether an account exists.
- Store verification state safely.
- Prevent token replay where applicable.
- Never log passwords or active authentication tokens.

5.8 AJAX and API Structure

Use predictable endpoint names:

```text
GET     /api/products/
GET     /api/products/1/
POST    /api/products/
PATCH   /api/products/1/
DELETE  /api/products/1/
```

Use meaningful HTTP status codes and consistent error structures.

Example:

```python
from django.http import JsonResponse


def health_check(request):
    return JsonResponse(
        {"status": "ok"},
        status=200,
    )
```

For a full REST API, Django REST Framework provides serializers, permissions, authentication, pagination, and other API tools.

5.9 Logging and Debugging

Avoid relying exclusively on `print()` in production.

Configure Django logging to route useful events to appropriate handlers.

Example:

```python
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
        },
    },
    "root": {
        "handlers": ["console"],
        "level": "INFO",
    },
}
```

Python logging example:

```python
import logging

logger = logging.getLogger(__name__)



def example_operation():
    logger.info("Example operation started")
```

Do not log secrets, passwords, authentication tokens, or unnecessary sensitive personal data.

5.10 Testing Strategy

Test:

- Model behavior and constraints.
- Form and serializer validation.
- Successful and unsuccessful requests.
- Authentication and authorization.
- Important business workflows.
- Payment and order edge cases.
- API response status codes.
- Permission boundaries.

Example:

```python
from django.test import TestCase
from django.urls import reverse



class StudentPageTests(TestCase):
    def test_student_page_returns_200(self):
        response = self.client.get(
            reverse("students:list")
        )

        self.assertEqual(response.status_code, 200)
```

This test assumes the named URL exists.

Run:

```bash
python manage.py test
```

```

## 5.11 Deployment Best Practices

A common production architecture is:

```text
Browser
   |
   | HTTPS
   v
Nginx
   |
   | Reverse Proxy
   v
Gunicorn / Django
   |
   +------> PostgreSQL
   |
   +------> Redis (optional)
```

Production checklist:

- Use a production WSGI or ASGI server appropriate to the application.
- Use Nginx or another suitable reverse proxy where needed.
- Configure HTTPS and certificates.
- Set `DEBUG=False`.
- Keep secrets outside source control.
- Run migrations through a controlled deployment process.
- Collect static files.
- Protect private media.
- Configure logs and monitoring.
- Back up the database and test restoration.
- Run deployment checks.
- Document rollback procedures.

Run:

```bash

```
python manage.py check --deploy
```

PART 6: Quick Revision Checklist

Before starting a Django project, verify the following:

Development

- [ ] Python version is supported.
- [ ] Virtual environment is active.
- [ ] Dependencies are recorded.
- [ ] Project and apps have clear responsibilities.
- [ ] URL names and namespaces are consistent.
- [ ] Templates use inheritance where helpful.
- [ ] Forms validate input.
- [ ] Tests cover important functionality.

Database

- [ ] Models define appropriate fields and constraints.
- [ ] Migrations are committed to source control.
- [ ] Relationships are modeled correctly.
- [ ] QuerySets avoid unnecessary database queries.
- [ ] Indexes are based on query requirements.
- [ ] Database backups and restoration are tested.

Security

- [ ] `DEBUG=False` in production.
- [ ] `SECRET_KEY` is protected.
- [ ] `ALLOWED_HOSTS` is configured.
- [ ] HTTPS is configured.
- [ ] CSRF protection is preserved.
- [ ] Authorization is enforced server-side.
- [ ] Uploaded files are handled safely.
- [ ] Dependencies are updated.

Deployment

- [ ] Production server is configured.
- [ ] Nginx or an equivalent proxy is configured where needed.
- [ ] Static files are collected and served correctly.
- [ ] Environment variables are configured.
- [ ] Logging and monitoring are available.
- [ ] Migrations are applied safely.
- [ ] Database backups and rollback procedures exist.

PART 7: Final Interview Questions

Q1. Why use a custom user model?

It allows a project to customize its user representation and reduces the need for difficult user-model changes later. Decide on it before the first migrations whenever possible.

Q2. What is the N+1 query problem?

It occurs when retrieving a collection causes additional queries for related objects, often one query per object. `select_related()` and `prefetch_related()` can reduce these queries.

Q3. What is the purpose of a service layer?

It can organize reusable business operations, especially when an operation spans multiple models or steps. It is an architectural choice, not a mandatory Django component.

Q4. What is the difference between session authentication and JWT?

Session authentication typically uses a server-managed session referenced by a cookie. JWT-based systems use signed tokens containing claims. Each approach has different storage, revocation, and security considerations.

Q5. Why use Nginx with Gunicorn?

Nginx can handle TLS termination, reverse proxying, and static-file delivery, while Gunicorn runs the Django WSGI application.

Q6. Why use automated tests?

Tests help detect regressions, verify business rules, and improve confidence when changing code.

Q7. Why should production secrets not be committed to Git?

Anyone with access to the repository may gain access to credentials. Use environment variables or a secrets manager and rotate exposed credentials.

Q8. What is the purpose of `python manage.py check --deploy`?

It runs Django's deployment-oriented system checks and reports configuration issues that may affect production security or deployment readiness.

Official References

- Django documentation: https://docs.djangoproject.com/en/6.0/
- Django deployment checklist: https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/
- Django authentication: https://docs.djangoproject.com/en/6.0/topics/auth/
- Django database optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Django testing: https://docs.djangoproject.com/en/6.0/topics/testing/
- Django static files: https://docs.djangoproject.com/en/6.0/howto/static-files/
- Django REST Framework: https://www.django-rest-framework.org/

- VS Code keyboard shortcuts: https://code.visualstudio.com/shortcuts/keyboard-shortcuts-windows.pdf
- VS Code Emmet: https://code.visualstudio.com/docs/languages/html#_emmet

End of VS Code Shortcut Cheatsheet and Django Best Practices.